DEVOPS PROJECT REPORT

ClinQ

Clinic Appointment Booking System

with an end-to-end DevOps pipeline

FastAPI | React | PostgreSQL | Docker | GitHub Actions | Trivy
Terraform | Kubernetes | Helm | Prometheus | Grafana

STUDENT
Somyajit Saha

COURSE
DevOps

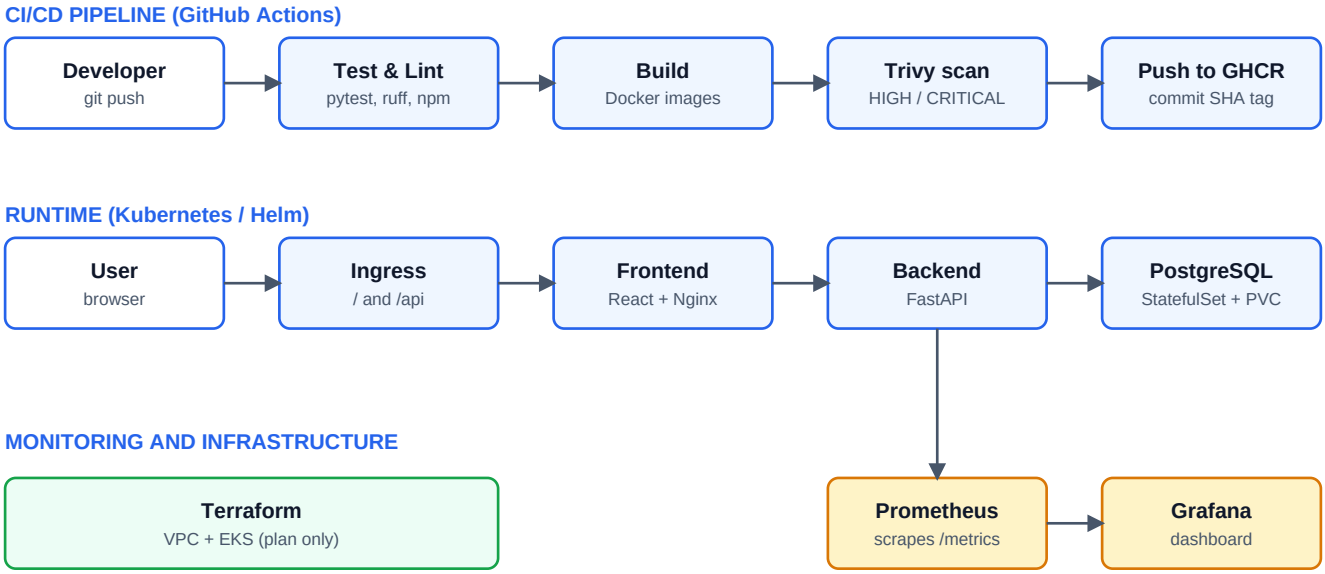
REPOSITORY
github.com/sergiocipher/project-clinq

Project Overview

ClinQ is a clinic appointment booking system. Staff can create, search, update and cancel appointments and see live counts of scheduled, completed and cancelled visits. The application is wrapped in a complete DevOps workflow: automated tests, containerization, a CI/CD pipeline with vulnerability scanning, infrastructure as code, Kubernetes deployment with Helm, and monitoring.

Scope: the Kubernetes deployment and monitoring were run on a local cluster. For AWS, Terraform was taken up to `terraform plan`; the EKS deployment was not performed.

Architecture



Technology Stack

Layer	Tools
Application	React + Vite (frontend), FastAPI (backend), PostgreSQL, Alembic migrations
Testing	pytest, SQLite test database, Ruff
Containers	Docker, Docker Compose, non-root images
CI/CD and security	GitHub Actions, GitHub Container Registry (GHCR), Trivy
Infrastructure	Terraform (AWS VPC and EKS, region ap-south-1)
Orchestration	Kubernetes, Helm, NGINX Ingress, Horizontal Pod Autoscaler
Monitoring	Prometheus, Grafana (kube-prometheus-stack)

1 Application

The backend is a FastAPI service with a PostgreSQL database whose schema is managed by Alembic. Each appointment stores the patient name, doctor, scheduled time, status (scheduled, completed, cancelled), priority (low, normal, urgent) and notes. The React dashboard shows summary cards, a searchable table with status filters, a recent-activity feed, and a build badge in the sidebar.

Method	Endpoint	Purpose
GET	`/api/appointments`	List, filter by status, search
POST	`/api/appointments`	Create an appointment
GET	`/api/appointments/stats`	Counts for the dashboard cards
GET / PUT / DELETE	`/api/appointments/{id}`	Read, update, delete one appointment
GET	`/health, /ready, /metrics`	Liveness, readiness (database check), Prometheus metrics

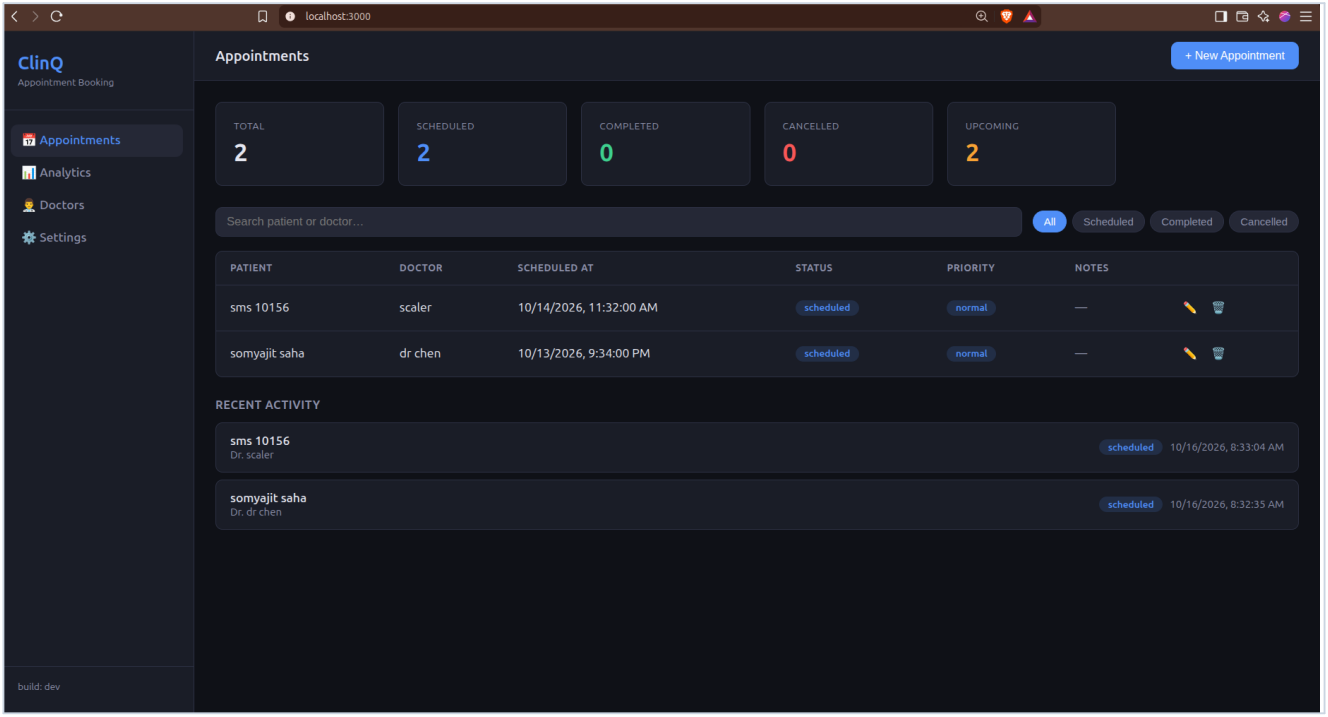


Figure 1: ClinQ dashboard served by the Docker Compose stack at localhost:3000.

2 Testing

The backend has **20 pytest tests** covering every endpoint: create, read, update and delete, input validation (422), search and status filters, statistics, health, readiness (including a simulated database failure returning 503) and metrics. Tests run against an in-memory SQLite database, so they are fast and never touch real data. All 20 tests pass.

```
(.venv) somyajit-saha@somyajit-saha-IdeaPad-Slim-3-15IRH8:~/project-clinq/backend$ pytest -v
/home/somyajit-saha/project-clinq/backend/.venv/lib/python3.12/site-packages/pytest_asyncio/plugin.py:208: PytestDeprecationWarning: The configuration option "asyncio_default_fixture_loop_scope" is unset.
The event loop scope for asynchronous fixtures will default to the fixture caching scope. Future versions of pytest-asyncio will default the loop scope for asynchronous fixtures to function scope. Set the default fixture loop scope explicitly in order to avoid unexpected behavior in the future. Valid fixture loop scopes are: "function", "class", "module", "package", "session"

  warnings.warn(PytestDeprecationWarning(_DEFAULT_FIXTURE_LOOP_SCOPE_UNSET))
============================================= test session starts ==============================================
platform linux -- Python 3.12.3, pytest-8.3.3, pluggy-1.7.0 -- /home/somyajit-saha/project-clinq/backend/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/somyajit-saha/project-clinq/backend
configfile: pytest.ini
testpaths: tests
plugins: anyio-4.15.1, asyncio-0.24.0
asyncio: mode=Mode.STRICT, default_loop_scope=None
collected 20 items

tests/test_appointments_api.py::test_create_returns_201 PASSED                                    [  5%]
tests/test_appointments_api.py::test_create_invalid_status_returns_422 PASSED                     [ 10%]
tests/test_appointments_api.py::test_create_missing_required_field_returns_422 PASSED             [ 15%]
tests/test_appointments_api.py::test_list_returns_all PASSED                                      [ 20%]
tests/test_appointments_api.py::test_list_status_filter PASSED                                    [ 25%]
tests/test_appointments_api.py::test_list_search_by_patient PASSED                                [ 30%]
tests/test_appointments_api.py::test_list_search_by_doctor PASSED                                 [ 35%]
tests/test_appointments_api.py::test_get_returns_200 PASSED                                       [ 40%]
tests/test_appointments_api.py::test_get_missing_returns_404 PASSED                               [ 45%]
tests/test_appointments_api.py::test_update_returns_200 PASSED                                    [ 50%]
tests/test_appointments_api.py::test_update_missing_returns_404 PASSED                            [ 55%]
tests/test_appointments_api.py::test_delete_returns_204 PASSED                                    [ 60%]
tests/test_appointments_api.py::test_delete_then_get_returns_404 PASSED                           [ 65%]
tests/test_appointments_api.py::test_stats_counts_all_statuses PASSED                             [ 70%]
tests/test_appointments_api.py::test_stats_empty_db PASSED                                        [ 75%]
tests/test_health.py::test_health_returns_ok PASSED                                               [ 80%]
tests/test_health.py::test_root_has_version PASSED                                                [ 85%]
tests/test_health.py::test_ready_returns_ready PASSED                                             [ 90%]
tests/test_health.py::test_ready_returns_503_when_db_broken PASSED                                [ 95%]
tests/test_health.py::test_metrics_contains_help_after_api_call PASSED                            [100%]
```

Figure 2: pytest output, 20 of 20 tests passed.

3 Containerization

Backend, frontend and PostgreSQL run together with Docker Compose. The frontend image is a multi-stage build (Node build stage, then unprivileged Nginx). Both application images run as non-root users with numeric IDs. PostgreSQL has a health check, and the backend applies Alembic migrations on startup.

```
(.venv) somyajit-saha@somyajit-saha-IdeaPad-Slim-3-15IRH8:~/project-clinq$ docker compose exec backend id
docker compose exec frontend id
uid=10001(app) gid=10001(app) groups=10001(app)
uid=101(nginx) gid=101(nginx) groups=101(nginx)
(.venv) somyajit-saha@somyajit-saha-IdeaPad-Slim-3-15IRH8:~/project-clinq$
```

Figure 3: backend runs as uid 10001 (app) and frontend as uid 101 (nginx), not root.

4 CI/CD Pipeline

A GitHub Actions workflow (`ci-cd.yml`) runs on every push to `main`:

- **Test & Lint**: Ruff, pytest and the frontend build must pass first.
- **Build / Trivy Scan / Push**: backend and frontend images are built, scanned, then pushed to GHCR tagged with the commit SHA (never `latest`).
- **Deploy to EKS**: optional job, skipped because no EKS cluster was provisioned.

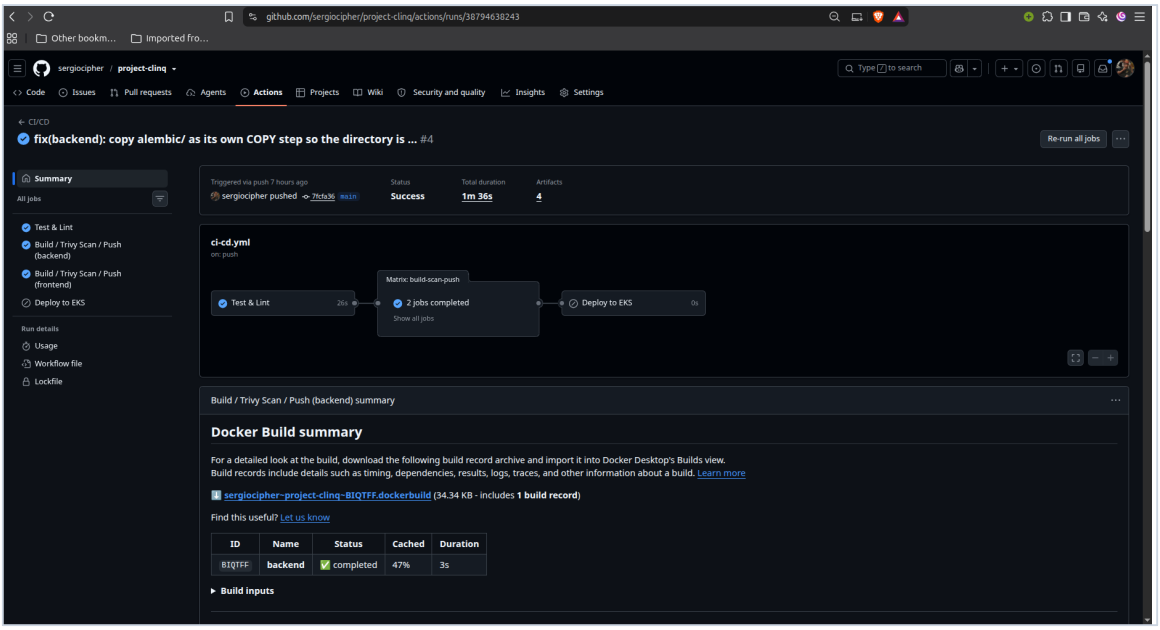


Figure 4: successful workflow run in 1m 36s; the optional EKS deploy job is skipped.

5 Security Scanning (Trivy)

Trivy scans each image for known vulnerabilities in OS packages and application dependencies. The step runs *before* the push and fails the job on HIGH or CRITICAL findings, so only scanned images reach the registry. Both the backend and frontend scans passed.

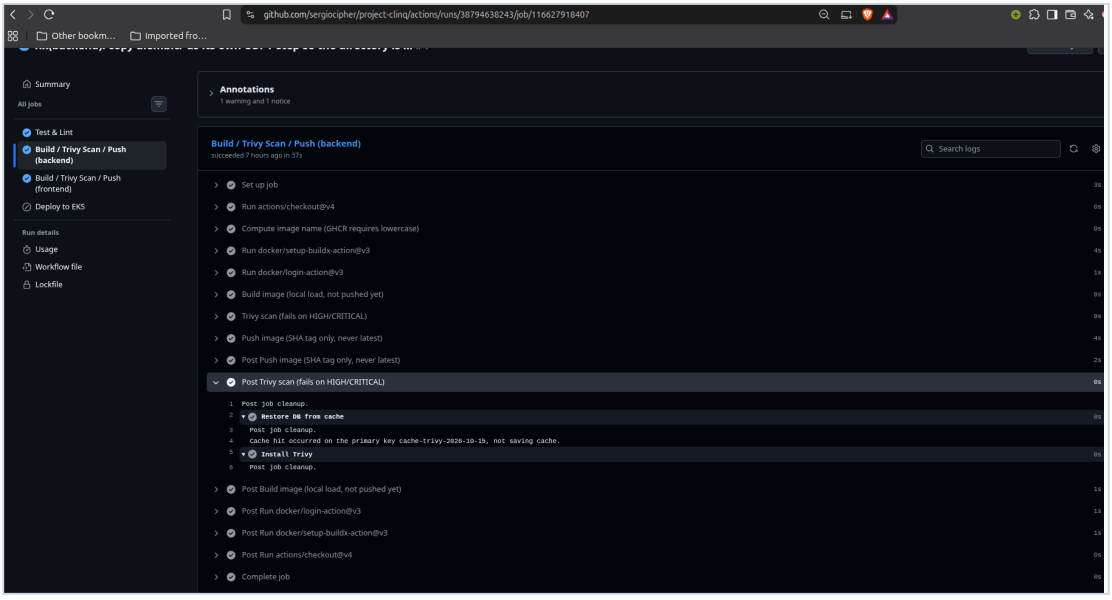


Figure 5: backend job; build, Trivy scan and push steps all succeed.

6 Infrastructure as Code (Terraform)

Terraform defines the AWS infrastructure in region **ap-south-1** using the community VPC and EKS modules: a VPC with two public and two private subnets, an EKS cluster named **project-clinq**, and a managed node group. The configuration is formatted, validated and planned. The plan creates **59 resources** (0 to change, 0 to destroy).

`terraform apply` was intentionally not run, to avoid AWS charges. The same Helm chart was verified on a local cluster instead (next section).

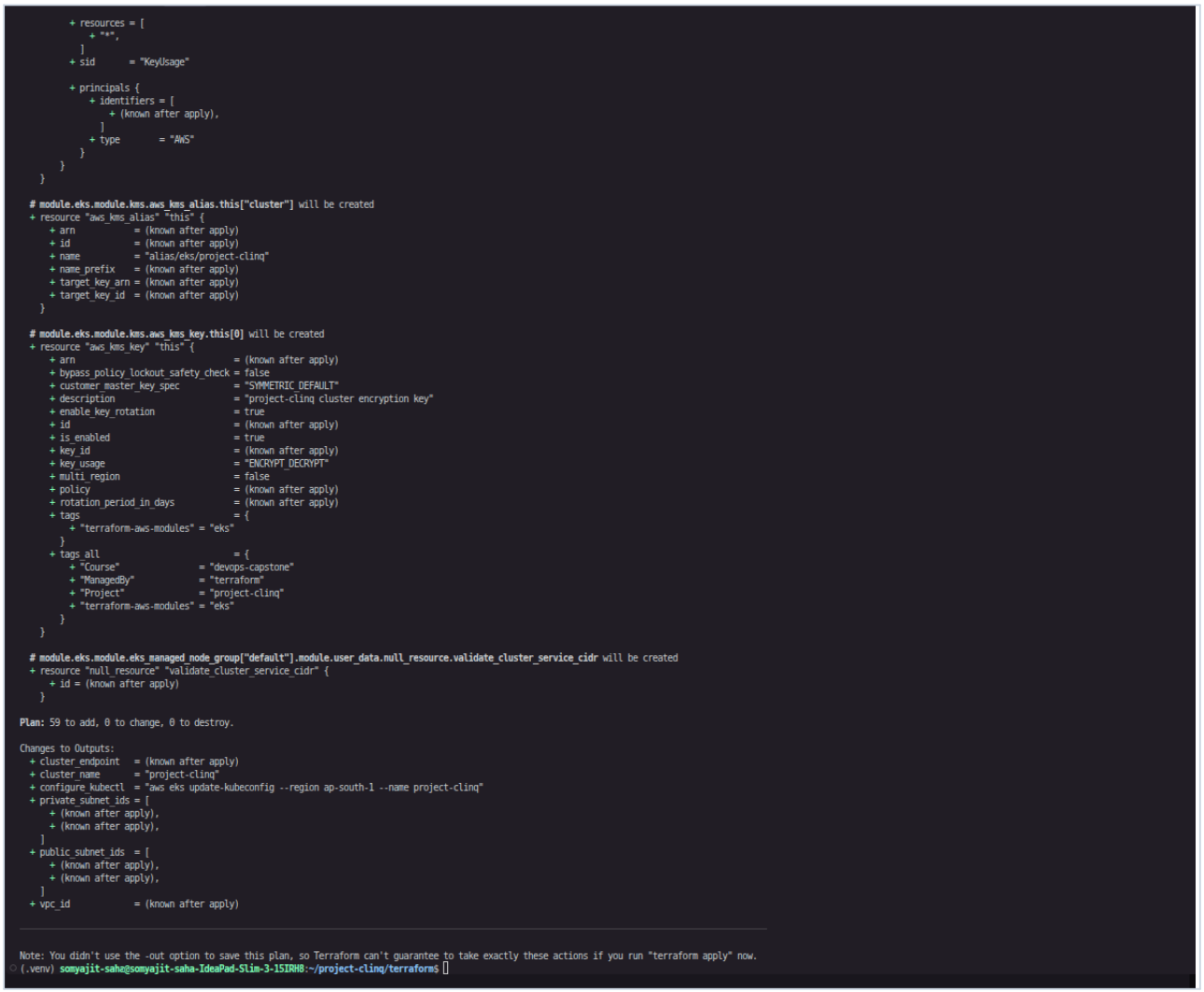


Figure 6: end of terraform plan: 59 to add, 0 to change, 0 to destroy, with cluster and subnet outputs.

7 Kubernetes and Helm

The application is packaged as the Helm chart **project-clinq** and was deployed to a local Kubernetes cluster with the development values file. Images are pulled from GHCR using the commit SHA tag.

Component	Configuration
Backend	Deployment with 2 replicas, ClusterIP Service, liveness (/health) and readiness (/ready) probes, non-root
Frontend	Deployment with 2 replicas, Nginx proxies /api to the backend Service, non-root
PostgreSQL	StatefulSet with persistent volume claim
Ingress	Routes / to the frontend and /api to the backend
Autoscaling	Horizontal Pod Autoscaler on the backend (CPU based)

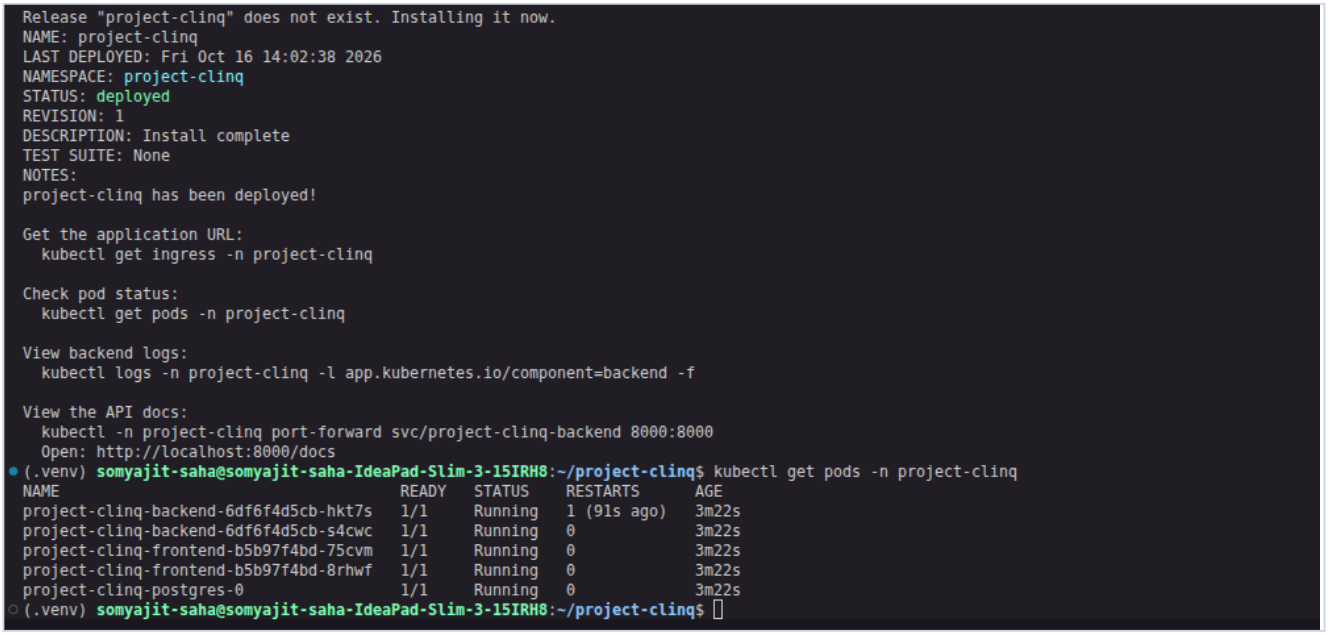


Figure 7: Helm release deployed; 2 backend, 2 frontend and 1 PostgreSQL pod all Running and Ready.

8 Observability

The backend exposes Prometheus metrics at `/metrics`. The kube-prometheus-stack chart provides Prometheus and Grafana, and a ServiceMonitor in the Helm chart lets Prometheus scrape the backend. A custom Grafana dashboard shows request rate and p95 latency per endpoint, 5xx error rate, total requests, backend pod CPU and current HPA replicas.

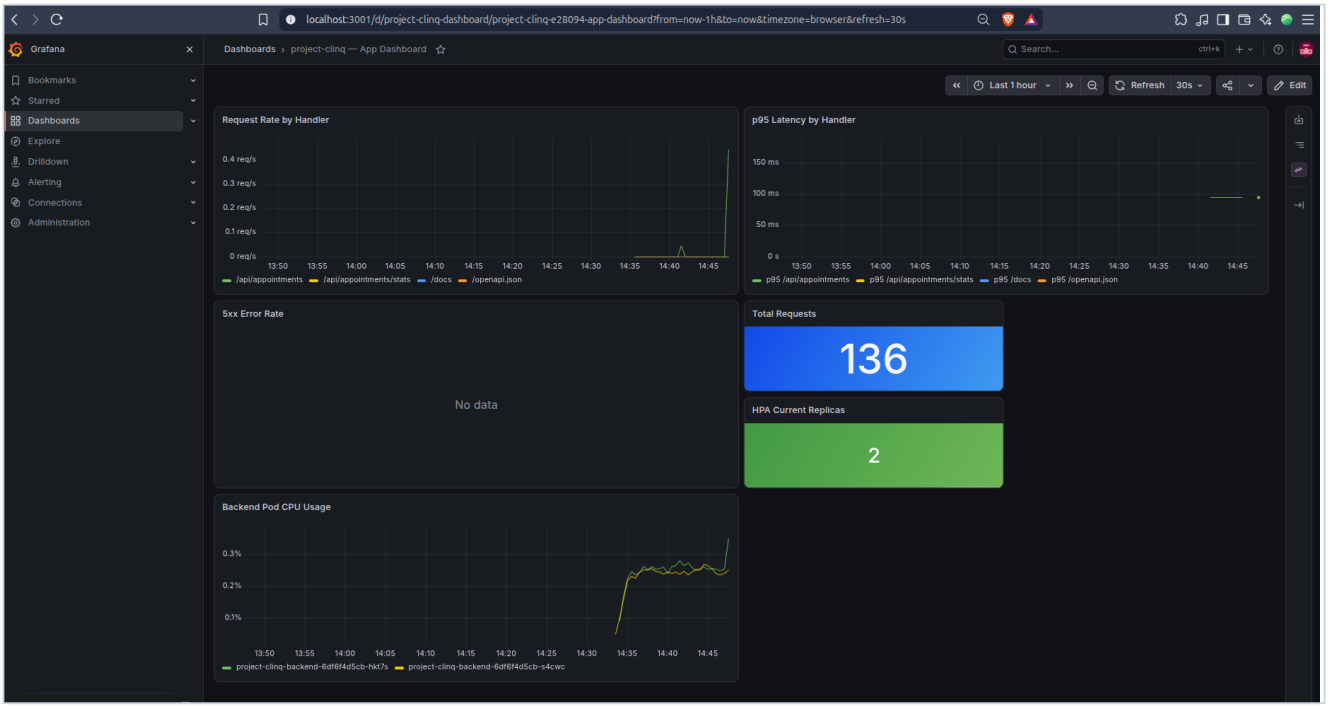


Figure 8: Grafana dashboard "project-clinq - App Dashboard" with live data: 136 total requests, 2 HPA replicas. The 5xx panel shows no data, meaning no server errors were recorded.